Three Case Studies from Ian Sommerville's "Software Engineering" Textbook (10th Edition)

Based on the 10th edition of Ian Sommerville's *Software Engineering*, the book introduces several case studies in Chapter 1 to illustrate key concepts in software engineering. These are used throughout the text to demonstrate real-world applications across different system types. Here are three selected case studies:

1. **Insulin Pump Control System**: This case study focuses on a medical device software system for controlling insulin delivery to diabetic patients. It highlights critical aspects of safety-critical systems, requirements engineering for reliability, and the challenges of embedded software in healthcare.
2. **Wildlife Tracking System**: This involves a system for monitoring and tracking endangered wildlife using sensor networks and GPS technology. It exemplifies data-intensive systems, emphasizing issues like real-time data processing, environmental monitoring, and integration of hardware-software components.
3. **Digital Learning Environment (iLearn)**: This case study describes an educational software platform designed for students aged 3 to 18 in schools. It covers user-centered design, agile development techniques, and scalability in web-based learning systems, including features for personalized learning and teacher-student interaction.

For each of these case studies, I have prepared a hypothetical Software Requirements Specification (SRS) document in the IEEE 830-1998 standard format. These are constructed based on the descriptions and examples provided in the textbook, adapted to fit the IEEE template. Note that these are illustrative and not direct excerpts, as the book uses the case studies to discuss concepts rather than providing full SRS documents. The SRS focuses on functional and non-functional requirements relevant to the system type.

Software Requirements Specification for Insulin Pump Control System

1. Introduction

1.1 Purpose

The purpose of this SRS is to define the software requirements for the Insulin Pump Control System, a safety-critical embedded software that regulates insulin delivery for diabetic patients. This document serves as a contract between the development team and stakeholders, ensuring the system meets medical safety standards while providing reliable automated insulin administration.

1.2 Scope

The system controls an insulin pump device, interfacing with a patient's blood glucose sensor to compute and deliver precise insulin doses. It includes monitoring, alerting, and logging functionalities but excludes hardware design or external medical database integration. The system will be deployed in portable medical devices and must comply with FDA regulations for Class III medical devices.

1.3 Definitions, Acronyms, and Abbreviations

- **BGM**: Blood Glucose Monitor
- **UI**: User Interface
- **SRS**: Software Requirements Specification
- **FDA**: Food and Drug Administration

1.4 References

- Sommerville, I. (2015). *Software Engineering, 10th Edition*. Pearson.
- IEEE Std 830-1998. *Recommended Practice for Software Requirements Specifications*.

1.5 Overview

This SRS is organized into sections covering overall description, specific requirements, and supporting information. Section 2 provides a high-level view, while Section 3 details functional and non-functional requirements.

2. Overall Description

2.1 Product Perspective

This system is part of a broader medical IoT ecosystem for diabetes management. It builds on existing insulin pump hardware but introduces advanced software for predictive dosing based on Sommerville's case study on safety-critical systems.

2.2 Product Functions

- Monitor blood glucose levels in real-time.
- Calculate and administer insulin doses using predefined algorithms.
- Alert users to anomalies (e.g., low battery, high glucose).
- Log dosage history for medical review.

2.3 User Classes and Characteristics

- **Patients**: Diabetic individuals (ages 10+), requiring simple UI for status checks.

- **Healthcare Providers**: Doctors/nurses, accessing logs via a companion app for configuration.
- **System Administrators**: Medical device technicians for firmware updates.

2.4 Operating Environment

- Embedded on ARM-based microcontroller in a battery-powered pump.
- Operates in temperatures 0-40°C; interfaces with Bluetooth for data sync.
- Real-time OS (e.g., FreeRTOS) with low power consumption.

2.5 Design and Implementation Constraints

- Must use C language for embedded efficiency.
- Compliance with ISO 26262 for functional safety.
- No internet connectivity to ensure privacy.

2.6 Assumptions and Dependencies

- Assumes accurate BGM sensor input; dependencies on hardware calibration.
- User has basic training on device operation.

3. Specific Requirements

3.1 External Interfaces

- **User Interfaces**: Touchscreen LCD for dose confirmation; audible alerts.
- **Hardware Interfaces**: Serial connection to BGM sensor; pump actuator control.
- **Software Interfaces**: API for logging to external database via Bluetooth.
- **Communication Interfaces**: None (offline operation).

3.2 Functional Requirements

- **FR1**: The system shall read blood glucose levels from the BGM every 5 minutes.
- **FR2**: The system shall compute insulin dose using the formula: Dose = (Current Glucose - Target) * Sensitivity Factor, where Target = 100 mg/dL.
- **FR3**: The system shall deliver the computed dose via pump activation within 30 seconds of computation.
- **FR4**: The system shall log each dose event with timestamp and glucose reading to internal memory.
- **FR5**: The system shall generate an alert (visual/audible) if glucose exceeds 300 mg/dL or drops below 70 mg/dL.

3.3 Performance Requirements

- Response time for dose calculation: < 1 second.

- Uptime: 99.99% in continuous operation for 24 hours.
- Throughput: Handle 288 readings per day (every 5 minutes).

3.4 Logical Database Requirements

- Internal non-volatile memory for 30 days of logs (up to 10,000 entries).
- Fields: Timestamp, Glucose Level, Dose Amount, User Confirmation.

3.5 Design Constraints

- Software must be modular for safety verification (e.g., separate modules for monitoring and actuation).
- Adhere to MISRA C guidelines for embedded safety.

3.6 Software System Attributes

- **Reliability**: Mean time between failures > 10,000 hours; fault-tolerant with redundant checks.
- **Availability**: System shall enter safe mode (stop delivery) on any fault.
- **Security**: Encrypt logs; prevent unauthorized dose changes.
- **Maintainability**: Modular code with < 5% coupling between modules.
- **Portability**: Compatible with similar ARM hardware.

3.7 Other Requirements

- **Safety**: System shall not deliver dose without user confirmation for amounts > 5 units.
- **Standards Compliance**: IEC 62304 for medical software lifecycle.

4. Supporting Information

- Appendices: Use case diagrams (e.g., patient monitoring flow) and risk analysis matrix from Sommerville's case study.

Software Requirements Specification for Wildlife Tracking System

1. Introduction

1.1 Purpose

This SRS outlines the requirements for the Wildlife Tracking System, a data collection and analysis software for monitoring endangered species using GPS collars. It ensures the system supports conservation efforts by providing accurate, real-time tracking data.

1.2 Scope

The system processes data from field sensors and GPS devices, visualizes animal movements on maps, and generates reports. It excludes sensor hardware manufacturing but includes web-based dashboard access. Based on Sommerville's case study for data-intensive systems.

1.3 Definitions, Acronyms, and Abbreviations

- **GPS**: Global Positioning System
- **API**: Application Programming Interface
- **SRS**: Software Requirements Specification

1.4 References

- Sommerville, I. (2015). *Software Engineering, 10th Edition*. Pearson.
- IEEE Std 830-1998. *Recommended Practice for Software Requirements Specifications*.

1.5 Overview

Section 2 describes the system context, and Section 3 specifies detailed requirements.

2. Overall Description

2.1 Product Perspective

This system integrates with environmental sensor networks, extending traditional wildlife monitoring tools as discussed in the textbook's data management examples.

2.2 Product Functions

- Collect and store GPS data from animal collars.
- Analyze movement patterns (e.g., migration routes).
- Provide web-based visualization and alerts for poaching risks.
- Generate compliance reports for conservation agencies.

2.3 User Classes and Characteristics

- **Field Researchers**: Biologists uploading data from remote sites.
- **Analysts**: Conservation experts querying data for patterns.

- **Administrators**: Managing user access and system backups.

2.4 Operating Environment

- Server-side: Cloud platform (e.g., AWS) with web frontend (HTML5/JS).
- Client-side: Browsers on desktops/laptops; mobile app for field data entry.
- Operates 24/7 with internet connectivity.

2.5 Design and Implementation Constraints

- Use Python/Django for backend; PostgreSQL for database.
- Open-source GIS libraries (e.g., Leaflet) for mapping.

2.6 Assumptions and Dependencies

- Assumes reliable GPS signal in field areas; depends on collar battery life > 6 months.

3. Specific Requirements

3.1 External Interfaces

- **User Interfaces**: Web dashboard with interactive maps; mobile app for data upload.
- **Hardware Interfaces**: USB/Bluetooth for collar data sync.
- **Software Interfaces**: REST API for third-party conservation tools.
- **Communication Interfaces**: HTTPS for secure data transmission.

3.2 Functional Requirements

- **FR1**: The system shall ingest GPS coordinates from collars at intervals of 15 minutes.
- **FR2**: The system shall store location data with metadata (e.g., animal ID, timestamp) in a spatial database.
- **FR3**: The system shall generate heat maps of animal movements using geospatial queries.
- **FR4**: The system shall alert users via email/SMS if an animal enters a restricted zone.
- **FR5**: The system shall export reports in PDF/CSV format for regulatory submission.

3.3 Performance Requirements

- Data ingestion rate: 1,000 points per hour.
- Query response time: < 2 seconds for map rendering.

- Scalability: Support 500 concurrent users.

3.4 Logical Database Requirements

- Spatial database with tables for Animals (ID, Species), Locations (Lat, Long, Time), and Alerts.
- Capacity: 1 million records per year.

3.5 Design Constraints

- Modular design with microservices for data processing and visualization.

3.6 Software System Attributes

- **Reliability**: 99.9% data accuracy; backup every 24 hours.
- **Availability**: High-availability cloud deployment.
- **Security**: Role-based access; data encryption in transit/rest.
- **Maintainability**: API documentation; automated testing coverage > 80%.
- **Portability**: Cloud-agnostic deployment.

3.7 Other Requirements

- **Privacy**: Anonymize human observer data.
- **Interoperability**: Support standard GIS formats (e.g., KML).

4. Supporting Information

- Appendices: Entity-relationship diagram and sample data flows from the case study.

Software Requirements Specification for Digital Learning Environment (iLearn)

1. Introduction

1.1 Purpose

This SRS specifies the requirements for iLearn, a web-based digital learning platform for school students aged 3-18. It aims to facilitate interactive, personalized education while supporting teachers in content management.

1.2 Scope

The system provides adaptive learning modules, progress tracking, and collaboration tools. It targets K-12 education and excludes hardware procurement. Drawn from Sommerville's new case study on educational systems.

1.3 Definitions, Acronyms, and Abbreviations

- **LMS**: Learning Management System
- **UI/UX**: User Interface/User Experience
- **SRS**: Software Requirements Specification

1.4 References

- Sommerville, I. (2015). *Software Engineering, 10th Edition*. Pearson.
- IEEE Std 830-1998. *Recommended Practice for Software Requirements Specifications*.

1.5 Overview

The document covers system overview in Section 2 and detailed requirements in Section 3.

2. Overall Description

2.1 Product Perspective

iLearn is an agile-developed LMS enhancing traditional classroom tools, focusing on user-centered design as per the textbook's agile examples.

2.2 Product Functions

- Deliver personalized lesson content based on student progress.
- Enable teacher-student interactions via forums and quizzes.
- Track and report learning analytics.
- Support multimedia content (videos, interactive simulations).

2.3 User Classes and Characteristics

- **Students**: Ages 3-18, accessing via tablets/PCs for self-paced learning.
- **Teachers**: Educators creating and assigning content.
- **Parents/Administrators**: Viewing reports and managing accounts.

2.4 Operating Environment

- Web-based (responsive design); hosted on cloud (e.g., Azure).
- Compatible with major browsers; iOS/Android apps for mobile access.

2.5 Design and Implementation Constraints

- Use React.js for frontend; Node.js backend.
- Adhere to WCAG 2.1 for accessibility.

2.6 Assumptions and Dependencies

- Assumes school internet access; depends on user devices meeting minimum specs (e.g., 4GB RAM).

3. Specific Requirements

3.1 External Interfaces

- **User Interfaces**: Intuitive dashboard with gamified elements for students.
- **Hardware Interfaces**: Webcam/mic for virtual classes (optional).
- **Software Interfaces**: Integration with Google Classroom API.
- **Communication Interfaces**: WebSockets for real-time chat.

3.2 Functional Requirements

- **FR1**: The system shall adapt lesson difficulty based on quiz performance (e.g., 80% correct advances level).
- **FR2**: The system shall allow teachers to upload and assign multimedia content to student groups.
- **FR3**: The system shall track student progress and generate weekly reports.
- **FR4**: The system shall facilitate discussion forums with moderation tools.
- **FR5**: The system shall support collaborative editing for group projects.

3.3 Performance Requirements

- Page load time: < 3 seconds.
- Concurrent users: 10,000 per school district.
- Quiz processing: Instant feedback.

3.4 Logical Database Requirements

- Tables for Users (Student/Teacher profiles), Content (Lessons, Quizzes), and Analytics (Scores, Time Spent).
- Capacity: 1 TB for multimedia storage.

3.5 Design Constraints

- Agile iterations with user feedback loops.

3.6 Software System Attributes

- **Reliability**: Auto-save progress every 5 minutes.
- **Availability**: 99.5% uptime during school hours.
- **Security**: GDPR-compliant; parental consent for data.
- **Maintainability**: CI/CD pipeline; code modularity.
- **Portability**: Cross-platform compatibility.

3.7 Other Requirements

- **Usability**: Age-appropriate UI (e.g., simple for young students).
- **Scalability**: Handle peak usage during exams.

4. Supporting Information

- Appendices: User stories and wireframes inspired by the case study.